

SPECTRA: A Traceability Ontology for the 3GPP RAN Standardization Process

Supplementary Appendix (Companion to ISWC 2026 Resources Track Submission)

Sihyeon Choi* and Junho Lee

System LSI Business, Device Solutions Division,
Samsung Electronics, South Korea
{shyun12.choi, junho515.lee}@samsung.com

** First and corresponding author.*

Scope. This document contains material trimmed from the main paper to comply with the ISWC 2026 Resources Track 15-page body limit. The content is verbatim what would have appeared as Appendices A–G of the manuscript, and is also reproducible from the released artifacts (ontology/spectra.ttl, shapes/spectra-core.shacl.ttl, cqs/spectra_cq_v1.0/cypher/, and the JSON files under validation/) shipped at <https://github.com/spectra-ontology/spec-trace> and on Zenodo (DOI: 10.5281/zenodo.20034872; minted upon acceptance). Cross-references prefixed *Appendix* (e.g., Appendix A, Appendix D) refer to sections within this document; section references without the *Appendix* prefix point back to the main paper.

Appendix

A Ontology TTL and SHACL Excerpts

Ontology TTL (illustrative excerpt). Figure 1 shows the `spectra:Tdoc` class declaration with one functional and one non-functional OP, adapted from `ontology/spectra.ttl` (English labels and the explanatory `#` comments are paper-side annotations; the full TTL with all comments and the PROV-O block lives at the cited path).

SHACL shapes (illustrative excerpt). The full `shapes/spectra-core.shacl.ttl` declares 8 `sh:NodeShapes` covering the lifecycle classes. Figure 2 shows the `Tdoc` and `CR` shapes excerpted from the released file (the file uses the local `tdoc:` prefix bound to `https://w3id.org/spectra#`; we use `spectra:` below for readability).

B Sample SPARQL Queries

Figures 3 and 4 render two of the seven use scenarios from §main paper §6 as portable SPARQL. The Cypher equivalents distributed with SpectraCQ v1.0 follow the same join structure but use Neo4j syntax. Identifier prefixes are abbreviated for the figure.

```

spectra:Tdoc a owl:Class ;
  rdfs:subClassOf prov:Entity ;
  rdfs:label "Technical Document"@en .

spectra:submittedBy a owl:ObjectProperty ;
  rdfs:comment "Multiple companies can co-submit." ;
  rdfs:domain spectra:Tdoc ; rdfs:range spectra:Company .

spectra:hasContact a owl:ObjectProperty , owl:FunctionalProperty ;
  rdfs:domain spectra:Tdoc ; rdfs:range spectra:Contact .

spectra:presentedAt a owl:ObjectProperty ;
  rdfs:domain spectra:Tdoc ; rdfs:range spectra:Meeting .
# Not declared owl:FunctionalProperty: re-tabling admits
# multiple Meeting values per Tdoc (data-quality appendix).

```

Fig. 1. Excerpt from `ontology/spectra.ttl`. `hasContact` is functional (a Tdoc has at most one responsible Contact); `submittedBy` and `presentedAt` are intentionally non-functional to admit co-submitting companies and re-tabled TDocs respectively.

```

spectra:TdocShape a sh:NodeShape ;
  sh:targetClass spectra:Tdoc ;
  sh:property [ sh:path spectra:tdocNumber ; sh:datatype xsd:string ;
    sh:minCount 1 ; sh:maxCount 1 ] ;
  sh:property [ sh:path spectra:presentedAt ; sh:class spectra:Meeting ] ;
  sh:property [ sh:path spectra:targetRelease ; sh:class spectra:Release ;
    sh:maxCount 1 ] .

spectra:CRShape a sh:NodeShape ;
  sh:targetClass spectra:CR ;
  sh:property [ sh:path spectra:modifies ; sh:class spectra:Spec ] ;
  sh:property [ sh:path spectra:belongsToCRPack ; sh:class spectra:CRPack ;
    sh:maxCount 1 ] .

```

Fig. 2. Excerpt from `shapes/spectra-core.shacl.ttl`. `tdocNumber` is required and unique per Tdoc; `targetRelease` is bounded above (a TDoc targets at most one Release). `presentedAt` carries no `sh:maxCount` because TDocs may be re-tabled at a subsequent meeting under the same identifier (Appendix G); `modifies` is multi-valued for paired stage-2/stage-3 CRs.

C Per-WG Class Coverage

Table 1 expands the per-WG class-coverage summary cited in §main paper §6.E1; the full per-class breakdown lives in `validation/per_wg_class_coverage.json`. Absences reflect the released RAN-WG snapshot and loader scope rather than schema gaps: e.g., the released RAN5 snapshot does not instantiate `Agreement`, `WorkingAssumption`, `Summary`, and `SessionNotes`.

Table 1. Per-WG class coverage of the released SPECTRA schema, showing classes absent in each non-RAN1 WG instantiation.

WG	Coverage	Classes absent (compared with RAN1’s 26)
RAN1	26/26 (100%)	—
RAN2	22/26 (84.6%)	Chart, Figure, SessionNotes, Summary
RAN3	23/26 (88.5%)	Chart, Figure, SessionNotes
RAN4	24/26 (92.3%)	SessionNotes, Summary
RAN5	20/26 (76.9%)	Agreement, Chart, Figure, SessionNotes, Summary, WorkingAssumption

```
# S1: TDoc lineage behind current text of TS 38.214 §5.1.3
SELECT ?tdoc ?meeting ?date WHERE {
  ?section spectra:belongsToSpec spectra:TS_38.214 ;
           spectra:sectionNumber "5.1.3" .
  ?cr      spectra:modifiesSection ?section ;
           spectra:presentedAt    ?meeting .
  ?res     spectra:madeAt          ?meeting ;
           spectra:references     ?tdoc .
  ?meeting spectra:meetingNumberInt ?mn .
} ORDER BY DESC(?mn)
```

Fig. 3. S1 multi-hop traceability in SPARQL (simplified for the listing; the production Cypher additionally constrains `AgendaItem`/`CRPack` context per the spine-join rationale in §main paper §4, and folds the `Agreement/WorkingAssumption`-promotion `OPTIONAL` branches into the single `spectra:references` step shown here). The Resolution–CR join is implementation-level via shared `Meeting`.

```
# S5: TS sections affected by TR 38.889 study, with impact type
SELECT ?spec ?section ?impactType WHERE {
  ?tr    a spectra:TechnicalReport ;
         spectra:trNumber "38.889" ;
         spectra:hasTRImpact ?ti .
  ?ti    spectra:impactType    ?impactType ;
         spectra:impactsSpec   ?spec ;
         spectra:impactsSection ?section .
}
```

Fig. 4. S5 TR-to-TS impact propagation in SPARQL (TR-anchored direction; the released `SpectraCQ` S5 query file uses the inverse TS-anchored direction via `impactOfTR`, which the ontology supports symmetrically). Each `TRImpact` carries an `impactType` from the closed set {`NewTS`, `NewSection`, `ExtendSection`, `NoChange`}.

D Cumulative CQ Regression Cases (Extended)

This appendix expands the three QA cases summarised in §main paper §5. Each entry indicates the phase transition, the schema or naming change introduced, the class of previously-passing CQs that would have silently regressed without the cumulative re-execution, and the resolution. Per-case CQ counts are not extracted from the development history; the cumulative pass record (`validation/cq_results.json`) reports the post-fix steady state.

- **Case 1 (Phase-2→3, naming inconsistency `MADEAT` vs `MADE_AT`).** Three relationship labels and three property names were harmonized. Multiple Phase-1–2 CQs that filtered on the relationship name returned empty after the introduction of Phase-3 schema until the naming was unified across loaders and ontology.
- **Case 2 (Phase-3 audit, meeting-ID referential integrity).** A subset of Phase-1–2 CQs that joined `Tdoc`→`Meeting`→`Resolution` regressed when meeting identifiers used inconsistent capitalisations across `CR/Resolution` loaders. Three targeted patches resolved the gap without changing the schema.
- **Case 3 (Phase-3→4, three schema refinements).** `releaseId` was removed as redundant once `targetRelease` was declared functional; `hasCR` was declared `owl:InverseFunctionalProperty` so that a `CR` could be unambiguously located from its `CR-pack`; and the base class `Decision` was

renamed to **Resolution** for terminological alignment with 3GPP chairman-note vocabulary. A further set of Phase-1–2 CQs that referenced the renamed class would have silently regressed.

Cumulative CQ counts at the four transitions were 59, 104, 119, and 137. Per-CQ pass/fail is recorded in `validation/cq_results.json`; the regressions surfaced by cumulative re-execution would not have been detected by single-phase validation before release.

E Reproducibility Checklist (Mapping to Release Files)

Table 2 expands the Reproducibility paragraph in §main paper §7 into a per-claim mapping. Each row links a public claim made in the paper to the file(s) under `release_package/` that support it and to the `verify_release.py` check that re-derives it. All checks pass on the v1.0.0 release (`tests/verify_release.py` exits 0).

Table 2. Public-claim → artifact → `verify_release.py` mapping.

Claim	Artifact	Check
Ontology has 886 triples	<code>ontology/spectra.ttl</code>	<code>verify_release.py</code> §1
Bundled instantiation snippet conforms to SHACL	<code>examples/instantiation_snippet.ttl</code> , <code>shapes/spectra-core.shacl.ttl</code>	<code>verify_release.py</code> §2
Process-KG union conforms to SHACL (~2.44M triples, Appendix G)	<code>examples/process_kg/*.ttl</code> , <code>shapes/spectra-core.shacl.ttl</code>	<code>verify_release.py</code> §2b
End-to-end SPARQL example returns the documented row	<code>examples/end_to_end/</code>	<code>verify_release.py</code> §3
SpectraCQ ships 137 CQ + 137 Cypher with 25/34/45/15/18 phase distribution and 137-PASS RAN1 verdict	<code>cqs/spectra_cq_v1.0/</code>	<code>verify_release.py</code> §4
Structural metrics (26/53/81/20/2/15/6/2 + 6 PROV-O axioms)	<code>validation/structural_metrics.json</code>	<code>verify_release.py</code> §5
All validation manifest references resolve	<code>validation/validation_manifest.md</code>	<code>verify_release.py</code> §6
SpectraCQ text and Cypher contain no real company names	<code>cqs/spectra_cq_v1.0/</code>	<code>verify_release.py</code> §7

The `examples/process_kg/` directory (LS/CR routing across RAN1–RAN5 + RAN1 TDoc structural metadata, body content stripped) is verified via SHACL conformance against `shapes/spectra-core.shacl.ttl`. Both `examples/process_kg/` and `cqs/spectra_cq_v1.0/` retain verbatim company names matching the public 3GPP TDOC_List.xlsx.

F Artifact Treatment, Attribution, and Redistribution

The release bundles four artifact tiers governed by different rules (Table 3). The treatment reflects *what the data is*: 3GPP-public metadata (including company names from TDOC_List.xlsx) is redistributed verbatim, illustrative TTL

uses synthetic identifiers, and 3GPP-derived body text is retained under explicit 3GPP attribution.

Table 3. Per-tier treatment, license/attribution, and rationale.

Artifact	Treatment	License / attribution	Rationale
<code>examples/process_kg/</code> (routing edges + RAN1 TDoc metadata)	verbatim metadata	CC-BY 4.0	Redistributes metadata already public on 3GPP’s per-meeting <code>TDOC_List.xlsx</code> and TDoc cover pages; redacting would discard recoverable information.
<code>cqs/spectra_cq_v1.0/</code> (NL questions, Cypher)	verbatim company names	CC-BY 4.0	Released as an executable CQ-to-query benchmark; company names match <code>TDOC_List.xlsx</code> so queries return real results on any 3GPP-conformant KG.
<code>examples/real_world_mini/</code> , <code>examples/end_to_end/</code> Body content (CR/TR/TS text, <code>discussionText values</code>)	synthetic identifiers verbatim	CC-BY 4.0 3GPP-derived; explicit 3GPP source attribution	Templates for instantiation; no real-world data is implied. Sentence-level rendered text retained with explicit 3GPP attribution, following the academic redistribution practice established by [?,?]. Original PDFs remain freely accessible at https://www.3gpp.org/ftp/ .

Company names. Both process-KG metadata and SpectraCQ retain verbatim company names matching the public 3GPP per-meeting `TDOC_List.xlsx` and TDoc cover pages. No anonymization is applied because the names are already public 3GPP metadata. The `Contact` class instances populated in our internal operational KG are excluded from the public release; the released TTLs contain no personal correspondence from document body text.

Two-tier licence boundary. SPECTRA-authored components (the OWL ontology, SHACL shapes, PyLODE documentation, SpectraCQ v1.0, sample SPARQL/Cypher queries, synthetic instantiation examples, sanitized parsing-pipeline source, and `verify_release.py`) are released under CC-BY 4.0. Body-text literals derived from 3GPP CR/TR/TS PDFs are redistributed with explicit 3GPP source attribution, following the academic redistribution practice established by [?,?].

Verifiable boundary. The `verify_release.py` script (Section 7) verifies the `real_world_mini` synthetic instantiation contains only synthetic identifiers (no real company-name leak from the operational KG into the synthetic example set).

G Process-KG Data Quality and SHACL Conformance

The cross-WG process KG (`examples/process_kg/`) is shipped as three union-compatible Turtle files: `ls_routing.ttl` (195K triples, 25,586 distinct LSs), `cr_routing.ttl` (1.25M triples, 192,967 CRs), and `ran1_tdoc_metadata.ttl`

(991K triples, 123,678 TDocs). Together they form a 2.44M-triple union graph that conforms to `shapes/spectra-core.shacl.ttl` with zero violations under `pyshacl`. Note: the schema graph (`ontology/spectra.ttl`, 886 triples) and this 2.44M-triple process KG measure different things—the 886 number counts *schema axioms* (class/property declarations), while 2.44M counts *instance triples* populated against that schema.

Three population-time conventions and one upstream-data observation, derived directly from inspection of the released TTLs and the source RAN1–RAN5 KGs, govern how the schema’s lifecycle classes are instantiated; readers querying the release should be aware of them:

- **LS originatedFrom for outgoing LSs is implicit.** In 3GPP practice, an outgoing Liaison Statement (`direction = "out"`) has the host Working Group as its source by definition — the originating WG is the one tabling the LS. The Phase-1 RAN loaders therefore record `originatedFrom` only for incoming LSs (`direction = "in"`; coverage 51.9–96.6% across RAN1–RAN5, depending on the availability of an explicit *Original LS* reference on the cover sheet). For outgoing LSs the host WG is recoverable from the LS’s hosting KG.
- **LS sentTo reflects upstream coverage.** 1,556 of 25,586 distinct LSs (5.8%) carry no `sentTo` edge in the released union, mirroring an empty *To*: cell on the source TDoc cover sheet. SHACL therefore does not assert `sh:minCount` on `LSShape.sentTo`; consumers requiring a complete recipient list should treat absence as upstream-metadata gap rather than schema violation.
- **Tdoc presentedAt can be multi-valued under re-tabling.** In the released RAN1 snapshot, 123,678 source metadata records resolve to 123,677 distinct TDoc identifiers because one TDoc was re-tabled at a subsequent meeting under the same identifier and so appears in the cover-sheet records of more than one Meeting. `TdocShape.presentedAt` therefore omits `sh:maxCount`; queries treating `presentedAt` as functional should add an `ORDER BY` or aggregate clause.
- **Source companies are kept verbatim throughout.** Both the process-KG (`examples/process_kg/`) and the SpectraCQ benchmark (`cqs/spectra_cq_v1.0/`) retain real company names matching the public 3GPP per-meeting `TDOC_List.xlsx` and TDoc cover pages. No anonymization is applied because the names are already public.

The corresponding SHACL shape relaxations (`LSShape.originatedFrom`, `LSShape.sentTo`, `CRShape.modifies`, `TdocShape.presentedAt: sh:maxCount` removed) are all annotated inline in `shapes/spectra-core.shacl.ttl` with the rationales above.

CR-pack TSG plenary linkage. CR-pack approval at the TSG plenary is currently captured by the literal `tsgMeeting` attribute on `CRPack` (`xsd:string`, e.g., “RAN#103”); a structural `approvedAt OP` linking `CRPack` to a `TSGPlenary` $\sqsubseteq$ `Meeting` subclass is staged for v1.1 to support *approved/postponed/rejected* CR-pack analytics directly in the schema.